

Pune Authors Association - Senior Engineer Interview Prep Guide

Q1: "You wrote you collaborated within a cross-functional team — specifically, which parts of this system did you personally build versus what teammates, third-party libraries, or external services were responsible for?"

Answer:

As the lead developer, I architected the monolithic stack (React/Express/Prisma) and built the core API routes and data filtering logic. I heavily utilized automated code generation to accelerate our frontend scaffolding. Meanwhile, I delegated specific feature verticals to my teammates—for example, Naitik built out our automated event notification system and Point-of-Sale UI logic.

For the heavy lifting, we didn't reinvent the wheel:

- **UI Components:** We relied on Radix UI and Tailwind CSS so we didn't waste time on accessibility or CSS-in-JS overhead.
- **Database:** Prisma ORM abstracted our SQL to prevent injection attacks and catch relational bugs at compile time.
- **Infrastructure:** Vercel/Amplify handled our deployment, allowing us to focus purely on business logic rather than Nginx or Linux server configuration.

Q2: "Walk me through the full data flow for one 'digital asset' — from an author uploading it, through your React storefront and Express API, to where and how it's persisted in PostgreSQL, and back out to a buyer."

Answer:

1. **Upload (React -> Express):** The author submits a `FormData` payload containing metadata and the cover image. It hits `POST /api/author/books`.
2. **Processing (Express + Multer + Sharp):** `verifyToken` checks auth. `multer` drops the binary file into `/uploads`. A custom function uses `sharp` to resize the image to 600x900, compress it to WebP, and converts it into a **Base64 string**. The temporary file is deleted.
3. **Persistence (Prisma -> PostgreSQL):** `prisma.book.create()` inserts the book. The massive Base64 string is stored directly in the `coverUrl` column.
4. **Display (Storefront):** When a buyer visits the catalogue, the API returns the JSON. React renders `<img src={book.coverUrl} />`, parsing the Base64 directly without a secondary HTTP image request.
5. **Checkout:** The buyer submits `POST /api/orders`. Prisma uses a transaction to create an `Order` and an `OrderItem` linked via Foreign Key to the `Book`.

Q3: "You used React 18, TypeScript, and Tailwind CSS for the storefronts and dashboards — why these specifically over, say, Next.js for the storefront or plain CSS/styled-components?"

Answer:

- **Why React 18 SPA over Next.js:** While Next.js is great for SEO, 80% of our platform's weight is behind an auth-wall (Author/Admin/POS Dashboards). For highly interactive, private dashboards, an SPA built with Vite provided a much simpler mental model (no Server Components/Hydration mismatches) and lightning-fast HMR for iteration.
 - **Why Tailwind over Styled-Components:** Tailwind pairs perfectly with our headless Radix UI primitives. It allowed us to inject styling without context-switching. Also, styled-components has runtime overhead, whereas Tailwind compiles statically. Crucially, AI generation models excel at writing Tailwind, speeding up our scaffolding.
 - **Why TypeScript:** We used TS on the frontend paired with Zod validation to infer types for complex onboarding forms, catching prop-drilling errors at compile time rather than runtime.
-

Q4: "Supporting 30+ live authors is the most concrete claim here — what in your schema or API design would break or need to change if that number were 10x larger, and how do you know?"

Answer:

1. **Base64 Images in Postgres:** Storing massive Base64 strings in the `coverUrl` column works for 30 authors. At 300+, `SELECT *` queries will pull hundreds of megabytes of text into Node.js memory, causing Out of Memory (OOM) crashes. I'd move images to AWS S3.
2. **Sequential Scans:** The `schema.prisma` relies purely on primary keys and lacks custom `@@index` directives on foreign keys. Dashboards filtering orders will trigger massive sequential scans, requiring B-Tree indexes to fix.
3. **API Timeouts on Emails:** Our event approval emails use synchronous delays to avoid SMTP bans. 300+ emails in a loop will cause the HTTP request to hit a 504 Gateway Timeout. This must be offloaded to a Redis queue.
4. **Local File Storage:** `multer` saves to the local `/uploads` disk. We cannot scale the Express backend to multiple instances behind a load balancer without migrating to stateless S3 storage.

Q5: "You say you implemented strict JWT role-based access control — describe exactly how a role is assigned, where the JWT is validated, and what happens if a token is stolen or a role check is bypassed."

Answer:

- **Assignment:** Roles (ADMIN, AUTHOR, CUSTOMER) are assigned at `/register`. The endpoint explicitly hardcodes a block preventing privilege escalation via POST data payload (`if (userRole === 'ADMIN') return 403`).
- **Validation:** React passes the token in the `Authorization: Bearer` header. The `verifyToken` middleware decodes it via `jwt.verify` and attaches `req.user`. A secondary `isAdmin` middleware checks `req.user.role === 'ADMIN'`.
- **Stolen Tokens:** Because the JWT expires in 30 days and we lack a database-backed session table or refresh token rotation, a stolen token grants 30 days of unimpeded access unless we rotate the global `JWT_SECRET`, which logs everyone out.

- **Bypassed Checks:** Since Postgres doesn't enforce Row Level Security (RLS) automatically here, if a developer forgets the `isAdmin` middleware on a route, any valid token (even a Customer) can execute the DB query.

Q6: "Walk me through what happens end-to-end if a POS transaction succeeds on the client but the Express server crashes before writing to PostgreSQL — how would you detect or prevent a lost or duplicate sale?"

Answer:

- **The Current State:** The POS creation and stock decrement are wrapped in a `prisma.$transaction`. If the server crashes *before* the DB write, Postgres rolls it back safely. If the DB commits but the server crashes *before sending the HTTP 200 OK* back, the React client sees an error.
- **The Danger:** Because the API doesn't require an idempotency key, the author will hit "Retry" on the UI, creating a duplicate `PosOrder` and decrementing stock twice.
- **The Prevention:** The frontend should generate a UUID (`idempotencyKey`) on click. The DB schema should enforce an `@unique` constraint on this key. On retry, Postgres rejects the duplicate, and the backend catches the error to return the already-completed order data.

Q7: "You built administrative dashboards surfacing author performance analytics via Recharts — where is that analytics data computed, and what happens to dashboard load time as the underlying dataset grows?"

Answer:

- **Where it's computed:** It uses **In-Memory Reduction on the Backend**. The API fetches raw arrays of `posOrders` via Prisma and loops over them using JavaScript `.reduce()` to aggregate totals before sending the data to Recharts.
- **The Breakdown at Scale:**
 1. **Node.js Memory:** Fetching 100,000 orders into Express and iterating over them in single-threaded JavaScript will block the event loop and freeze the

server. We must push the math down to the database using SQL aggregations (GROUP BY).

2. **Network Payload:** Sending 100,000 JSON logs to the frontend will cause massive payload bloat. Data must be time-bucketed (e.g., grouped by month) on the backend first.
3. **Recharts SVG Overload:** Recharts uses DOM SVG nodes. Drawing 10,000 points will crash the browser renderer. We would need to migrate to Canvas/WebGL-based libraries like Apache ECharts for massive datasets.

Q8: "How did you actually verify the REST API and JWT access control worked correctly before calling it secure — what tests, tools, or manual checks did you run, and what's one bug you caught this way?"

Answer:

- **The Testing Approach:** Instead of an exhaustive Jest suite for the MVP, I built modular, manual Node.js scratch scripts (`test-api.js`, `test_logic.js`). I used native `fetch()` to hammer endpoints with mock JSON payloads and manually swapped out JWTs to ensure the `isAdmin` middleware threw a `403 Forbidden` for Authors trying to hit Admin endpoints.
- **The Bug Caught:** The most significant bug was an infinite redirect loop on the frontend. I intentionally simulated a token expiration. The backend correctly threw a `401 Unauthorized`, but React mishandled the state. It redirected to Login, but Login saw a stale token in `localStorage` and immediately bounced the user back to the Dashboard, causing an infinite loop. I had to explicitly clear `localStorage` on any `401` response to fix it.

Q9: "Robust PostgreSQL database — what specific choice did you make in your schema or query design (e.g., normalization, indexing, transactions) that you'd call a deliberate trade-off, and what alternative did you reject?"

Answer:

- **The Trade-off (JSON Columns):** In the `Author` schema, I deliberately broke strict 3rd Normal Form (3NF) by storing qualifications and hobbies as `json` columns rather than creating dedicated relational tables.
- **Why:** I rejected strict normalization to avoid "migration hell." Our onboarding requirements evolved rapidly. JSON allowed extreme flexibility to add fields via the frontend without running database migrations, at the cost of making aggregate SQL searches harder.
- **The Trade-off (Zero Custom Indexes):** I relied entirely on default Primary Keys and avoided custom `@@index` B-Trees. I rejected premature optimization, accepting sequential scans on dashboards in the short term to ensure lightning-fast writes and frictionless migrations during the MVP.
- **The Trade-off (Transactions):** I used strict `prisma.$transaction` locks for POS checkout, choosing absolute ACID data integrity over maximum write-concurrency (throughput).

Q10: "Knowing what you know now, what would you redesign in this architecture — for example around the automated system notifications or the storefront/dashboard split — and why?"

Answer:

1. **Notifications (Event-Driven):** I would rip the synchronous `nodemailer` and `setInterval` cron jobs out of the Express API. I'd implement a message broker (Redis/BullMQ) so the API instantly returns a `200 OK` while a background worker handles SMTP dispatches, retries, and rate-limiting.
2. **Storefront Split (Next.js):** I would cleave the Vite SPA monolithic architecture. The public-facing storefront requires SEO and fast Time-to-Interactive, so I'd rebuild it in Next.js (using Server-Side Rendering/SSG). The private, auth-walled dashboards would remain a React SPA on a subdomain.
3. **Database Bloat (Object Storage):** I would immediately rip the Base64 image strings out of the PostgreSQL `coverUrl` columns and implement an AWS S3 pipeline, storing only the CDN URL in the database to prevent memory crashes.